
ORRA

Final Prototype v3.0 Restoration & Rebuild Guide

Complete Build Snapshot for Mainstream Launch

Git Tag: v3.0-final-prototype
Commit: 5517804
Date: May 05, 2026

Settings | Daily Digest | Holographic Toggles | Pulse Feed
Wellness Dashboard | Game Arena | Token Marketplace | Prism
Reels

This document contains everything needed to rebuild the ORRA application from scratch - source inventory, configs, database, deployment steps, and verified restoration procedures.

Table of Contents

1. Project Overview	4
2. Feature Inventory	4
3. Source File Inventory	5
3.1 API Routes	6
3.2 UI Components (src/components/aura/)	7
3.3 Game Components (src/components/aura/games/)	8
3.4 Store & Libraries	8
4. Environment & Dependencies	9
4.1 Environment Variables	9
4.2 Key Dependencies	9
5. Database Schema	10
6. Configuration Files	11
7. Build & Deployment Procedure	12
7.1 Build Command Sequence	12
7.2 Critical Build Notes	12
7.3 Server Start Script (start-server.sh)	13
8. Backup Archive Contents	14
9. Full Restoration Procedure	15
9.1 Fresh Environment Setup	15

9.2 Git-Based Restoration	15
9.3 Post-Restoration Verification	16
10. Critical Patterns & Known Gotchas	16
10.1 Static File Copying (CRITICAL)	16
10.2 Zustand localStorage Rehydration	17
10.3 NavView Routing Pattern	17
10.4 Server Restart Sequence	17
10.5 Prisma Schema and Database Sync	17
10.6 Caddy Configuration	18
11. Test Account Credentials	18
12. Git Reference	18
13. Quick Restore from Git Tag	19

1. Project Overview

ORRA is a futuristic dark-mode social media super-application built on Next.js 16 with a comprehensive feature set designed for mainstream social engagement. The platform integrates real-time social interactions, gamification through an Aura Token economy, wellness tracking, AI-powered companionship via Prism, and immersive content experiences including short-form video reels and dance challenges. This restoration guide captures the complete state of the v3.0 Final Prototype, the designated build for mainstream launch, ensuring that every component, configuration, and data artifact can be faithfully reproduced at any time.

Metric	Value
Framework	Next.js 16.1.x (standalone output)
Language	TypeScript / React 19
Database	SQLite via Prisma ORM 6.11.x
State Management	Zustand 5.x with localStorage persistence
UI Components	shadcn/ui + Radix UI + Tailwind CSS 4
Runtime	Node.js / Bun, PM2 process manager
Reverse Proxy	Caddy on port 81 -> localhost:3000
Git Tag	v3.0-final-prototype (commit 5517804)
Total Source Files	120+ across API routes, components, stores, hooks

Table 1: Core Project Metrics

The application follows a client-side routing architecture using Zustand store NavView state rather than traditional file-based routing. This allows instant view transitions without page reloads, creating a native app-like experience within the browser. Authentication is handled via NextAuth.js with credentials-based login, and all API routes are protected with session validation middleware. The Aura Token economy drives engagement through a sophisticated anti-farming tracking system that awards tokens and XP for verified interactions such as posting, liking, following, and maintaining daily streaks.

2. Feature Inventory

The following is a complete inventory of all features present in the v3.0 Final Prototype. Each feature maps to specific source files and components documented in subsequent sections. This inventory serves as both a feature checklist for verification during restoration and a reference for understanding the scope of the application at this milestone.

Feature	Description	Component
Pulse Feed	Social feed with posts, images, vibe tags, like/comment/share/repost/save actions	<code>pulse-feed.tsx</code>
Explore	Discovery page with search, trending tags, and suggested users	<code>explore.tsx</code>
Create Post	Modal for text/image posts with vibe tag selection	<code>create-post-modal.tsx</code>
Profile	User profile with avatar, cover, bio, stats, edit capability, founder badge	<code>profile.tsx</code>
Settings	Full settings page with holographic toggles for notifications, privacy, appearance	<code>settings.tsx</code>
Daily Digest	Personalized morning briefing with streak, tokens, trending, and quick stats	<code>daily-digest.tsx</code>
Notifications	Real-time notification feed with type-based icons and read tracking	<code>activity.tsx</code>
Messages	Direct messaging with chat creation, real-time updates, and shared posts	<code>messages.tsx</code>
Prism Reels	Short-form video viewer with categories, like/comment, remix support	<code>prism-reels.tsx</code>
Wellness	Wellness dashboard with mood tracking, breathing exercises, mindfulness	<code>wellness-dashboard.tsx</code>
Game Arena	10 mini-games including Trivia Blast, Meme Lab, Emoji Quest, Poll Party	<code>game-arena.tsx</code>
Token Marketplace	Aura Token economy with earn/spend tracking and level progression	<code>token-marketplace.tsx</code>
Prism Companion	AI chat companion with contextual responses	<code>prism-companion.tsx</code>
Dance Challenge	Challenge entries with voting and prize tracking	<code>dance-challenge.tsx</code>
Hubs	Community hubs with membership, posts, and online tracking	<code>hub.tsx</code>
Share Modal	Cross-platform sharing with DM integration	<code>share-modal.tsx</code>

Table 2: Complete Feature Inventory

3. Source File Inventory

All source files are located under the project root directory. The following inventory catalogs every critical file by category, providing a complete map of the application structure. During restoration, every

file listed here must be present for the application to function correctly. The git tag v3.0-final-prototype guarantees that the exact state of all these files is recoverable.

3.1 API Routes

API Route	Purpose
/api/auth/[...nextauth]	NextAuth.js authentication handler
/api/auth/login	Credentials-based login endpoint
/api/auth/logout	Session destruction and logout
/api/auth/signup	New user registration with handle uniqueness check
/api/me	Current authenticated user profile data
/api/posts	CRUD operations for Pulse Feed posts
/api/posts/[id]	Individual post operations (delete, update)
/api/comments	Comment creation on posts
/api/likes	Unified like system (posts, reels, dance entries, hub posts)
/api/follows	Follow/unfollow with follower/following counts
/api/reposts	Echo (repost) functionality
/api/saves	Bookmark/save posts and reels
/api/stories	Story creation and retrieval (24h expiry)
/api/reels	Prism Reels CRUD with category and view tracking
/api/dance/entries	Dance challenge entry submission
/api/dance/vote	Vote on dance challenge entries
/api/hubs/[hubId]	Hub details, membership, and posts
/api/chats	Chat creation and listing
/api/chats/[chatId]/messages	Message history and sending
/api/notifications	Notification feed with read/unread tracking
/api/users/[id]	User profile data retrieval
/api/users/profile	Profile update (bio, avatar, cover, handle)
/api/search	Unified search across users, posts, hubs
/api/orra/streak	Daily streak calculation and update
/api/polls/create	Poll creation with configurable duration

/api/polls/vote	Poll voting with uniqueness enforcement
/api/uploads	File upload and serving (images, videos)

Table 3: API Route Inventory

3.2 UI Components (src/components/aura/)

File	Role
page.tsx (app)	Main app shell, view routing, Daily Digest integration
layout.tsx (app)	Root layout with SessionProvider
globals.css (app)	Global styles, holographic animations, founder badge CSS
app-wrapper.tsx	Authenticated/unauthenticated app state management
auth-page.tsx	Login/signup interface with ORRA branding
sidebar.tsx	Left navigation with all view links including Settings
right-sidebar.tsx	Trending and suggested users panel
top-header.tsx	Top bar with search, notifications, and profile menu
pulse-feed.tsx	Main social feed with infinite scroll and vibe tags
explore.tsx	Discovery and search interface
create-post-modal.tsx	Post creation with image upload and vibe tagging
profile.tsx	User profile with founder badge animations
edit-profile-modal.tsx	Inline profile editing (bio, avatar, cover)
settings.tsx	Full settings with holographic toggles (6 sections)
daily-digest.tsx	Personalized daily briefing (auto-show once/day)
activity.tsx	Notification center with type-based icons
messages.tsx	Direct messaging with chat creation
prism-reels.tsx	Short-form video viewer and creator
reel-viewer.tsx	Full-screen reel playback with interactions
wellness-dashboard.tsx	Mood tracking, breathing, and mindfulness
game-arena.tsx	Game hub with 10 mini-game launchers
token-marketplace.tsx	Aura Token earn/spend and level tracking
prism-companion.tsx	AI companion chat interface

dance-challenge.tsx	Dance challenge entry and voting
hub.tsx	Community hub with membership and posts
share-modal.tsx	Cross-platform sharing with DM integration
vibe-check-modal.tsx	Vibe tag selection for posts
profile-setup-modal.tsx	First-login profile setup wizard
story-viewer.tsx	Full-screen story viewing with progress

Table 4: Aura Component Inventory

3.3 Game Components (src/components/aura/games/)

File	Game Description
aura-quiz.tsx	Multi-topic quiz with timed answers
dance-off.tsx	Dance challenge voting game
emoji-quest.tsx	Emoji-based puzzle adventure
guess-the-vibe.tsx	Vibe prediction game
hot-take.tsx	Opinion-based social game
meme-lab.tsx	Meme creation and rating
poll-party.tsx	Social polling game
prism-personality.tsx	Personality assessment quiz
trivia-blast.tsx	Rapid-fire trivia challenges
would-you-rather.tsx	Choice-based social game

Table 5: Game Component Inventory

3.4 Store & Libraries

File	Purpose
src/stores/orra-store.ts	Zustand store: NavView routing, settings state, digest tracking, user session
src/store/aura-store.ts	Legacy Aura store (supplementary state)
src/lib/auth.ts	NextAuth configuration with credentials provider
src/lib/auth-helpers.ts	Password hashing, session utilities

src/lib/db.ts	Prisma client singleton
src/lib/api-hooks.ts	React Query hooks for API data fetching
src/lib/use-current-user.ts	Current user hook with session-based auth
src/lib/data.ts	Seed data and mock data generators
src/lib/utils.ts	Utility functions (cn, formatters)
src/hooks/use-mobile.ts	Mobile detection hook
src/hooks/use-toast.ts	Toast notification hook
src/components/providers/session-provider.tsx	NextAuth session provider wrapper

Table 6: Store & Library Inventory

4. Environment & Dependencies

The ORRA application requires a specific set of runtime dependencies and environment variables to function correctly. This section documents the exact versions, configuration files, and environment setup needed to restore the application to a working state. All dependencies are locked in the package.json and bun.lock files, which are included in the backup archive.

4.1 Environment Variables

Variable	Value	Purpose
DATABASE_URL	file:/home/z/my-project/db/custom.db	SQLite database path
NEXTAUTH_SECRET	aura-super-secret-key-2027-dev-only	Session encryption key
NEXTAUTH_URL	http://localhost:3000	Auth callback base URL
AUTH_TRUST_HOST	true	Trust reverse proxy headers
NODE_ENV	production	Production mode flag
PORT	3000	Server listen port

Table 7: Environment Variables

4.2 Key Dependencies

Package	Version	Role
---------	---------	------

next	^16.1.1	React framework with standalone output
react	^19.0.0	UI library
prisma	^6.11.1	ORM and database toolkit
next-auth	^4.24.11	Authentication system
zustand	^5.0.6	Client state management
@tanstack/react-query	^5.82.0	Server state and data fetching
framer-motion	^12.23.2	Animation library
lucide-react	^0.525.0	Icon system
tailwindcss	^4	Utility-first CSS
recharts	^2.15.4	Chart and data visualization
sharp	^0.34.3	Image processing
zod	^4.0.2	Schema validation
better-sqlite3	^12.9.0	SQLite driver for Prisma

Table 8: Key Dependencies

5. Database Schema

ORRA uses SQLite as its database engine, managed through Prisma ORM. The database file is located at db/custom.db (referenced by DATABASE_URL). The schema includes 20+ models covering users, social interactions, gamification, messaging, and content management. A complete backup of the database is stored in the backups/final-prototype-20260505/ directory. The Prisma schema file at prisma/schema.prisma defines the complete data model and must be used with prisma db push to recreate the database structure.

Model	Key Fields	Purpose
User	email, handle, auraTokens, auraLevel, dailyStreak, badges	Core user profile with gamification
Post	text, images, vibeTag, likesCount, commentsCount	Pulse Feed posts
Comment	text, postId, authorId	Post comments
Like	userId, targetId, targetType	Unified like system (polymorphic)
Follow	followerId, followingId	User follow relationships

Reel	title, thumbnail, videoUrl, views, category	Prism short-form videos
Story	image, viewed, expiresAt	24-hour stories
Chat / DirectMessage	chatId, text, imageUrl, reaction	Direct messaging system
Notification	action, type, read, triggeredByUserId	Multi-type notification system
TokenAction	action, targetId, tokensEarned, xpEarned	Anti-farming token tracking
DanceChallenge / Entry	name, hashtag, prize, likesCount	Dance competition system
Hub / HubMember / HubPost	name, membersCount, onlineCount	Community hub system
Poll / PollOption / PollVote	question, duration, expiresAt	Poll creation and voting
Repost	userId, postId	Echo (repost) functionality
Save	userId, targetId, targetType	Bookmark system
Account / Session	provider, sessionToken, expires	NextAuth data models

Table 9: Database Model Inventory

6. Configuration Files

The following configuration files control the build, runtime, and deployment behavior of the ORRA application. Each file is critical to the correct operation of the system and must be present in the project root or its designated subdirectory. All files are included in both the git repository (tag v3.0-final-prototype) and the backup archive at backups/final-prototype-20260505/.

File	Location	Critical Settings
.env	Project root	DATABASE_URL, NEXTAUTH_SECRET, NEXTAUTH_URL
next.config.ts	Project root	output: standalone, serverActions bodySizeLimit: 50mb, rewrites for uploads/images
tailwind.config.ts	Project root	Dark mode class strategy, custom animations, shadcn/ui theme tokens
tsconfig.json	Project root	Path aliases (@/ -> src/), strict mode enabled
postcss.config.mjs	Project root	@tailwindcss/postcss plugin
components.json	Project root	shadcn/ui component configuration
Caddyfile	Project root	Port 81 reverse proxy to localhost:3000 with header forwarding

<code>ecosystem.config.cjs</code>	Project root	PM2 process config: orra-next (3001) + orra-proxy (3000)
<code>start-server.sh</code>	Project root	Production startup: kill old, copy static, exec node standalone
<code>package.json</code>	Project root	Build script: next build + cp -r static/standalone + cp -r public/standalone
<code>prisma/schema.prisma</code>	prisma/	SQLite provider, 20+ models, indexes, unique constraints

Table 10: Configuration File Inventory

7. Build & Deployment Procedure

This section provides the exact sequence of commands and operations required to build and deploy the ORRA application from source. The build process leverages Next.js standalone output mode, which produces a self-contained server bundle that does not require the full `node_modules` directory at runtime. This significantly reduces deployment size and improves startup time. The critical build step involves copying static assets and public files into the standalone directory, as the standalone build does not include these by default.

7.1 Build Command Sequence

Step	Command	Purpose
1	<code>cd /home/z/my-project</code>	Navigate to project root
2	<code>npm install (or bun install)</code>	Install all dependencies from package.json
3	<code>npx prisma generate</code>	Generate Prisma client from schema
4	<code>npx prisma db push</code>	Sync database schema (create tables if needed)
5	<code>npm run build</code>	Build Next.js standalone output + copy static/public
6	<code>bash start-server.sh</code>	Kill old process, copy fresh static, start server

Table 11: Build Command Sequence

7.2 Critical Build Notes

1. Static file copying MUST use `rm -rf + cp -r` (NOT `cp -rn`). The `-rn` flag (no-clobber) skips existing files, leaving stale chunks from previous builds that cause rendering failures. Always clear the target directory first.

2. The standalone server.js runs on the PORT environment variable (default 3000). It serves both the Next.js application and API routes. The server must be killed (fuser -k 3000/tcp) before restarting to avoid port conflicts.
3. Caddy acts as a reverse proxy on port 81, forwarding all traffic to localhost:3000. This provides HTTPS termination and clean URL access. The Caddyfile is located in the project root and must be loaded by the system Caddy service.
4. PM2 is configured but not required for single-instance deployment. The ecosystem.config.cjs defines two apps: orra-next (port 3001) and orra-proxy (port 3000). For simple deployments, start-server.sh is sufficient.
5. The build script in package.json runs: next build && cp -r .next/static .next/standalone/.next/ && cp -r public .next/standalone/. This is the only supported build pipeline. Modifying it may result in missing static assets.

7.3 Server Start Script (start-server.sh)

The production start-server.sh script handles the complete lifecycle: killing stale processes, refreshing static assets with rm -rf + cp -r (the critical pattern that prevents stale build artifacts), and launching the standalone Node.js server with all required environment variables. This script is the recommended way to start the application in production.

```
#!/bin/bash
cd /home/z/my-project
export PORT=3000
export DATABASE_URL="file:/home/z/my-project/db/custom.db"
export NEXTAUTH_SECRET="aura-super-secret-key-2027-dev-only"
export NEXTAUTH_URL="http://localhost:3000"
export AUTH_TRUST_HOST=true
export NODE_ENV=production

fuser -k 3000/tcp 2>/dev/null # Kill existing server
sleep 1

# Critical: rm -rf then cp -r (never cp -rn)
mkdir -p .next/standalone/.next/static
rm -rf .next/standalone/.next/static/*
cp -r .next/static/* .next/standalone/.next/static/
rm -rf .next/standalone/public
cp -r public .next/standalone/public
exec node .next/standalone/server.js
```

8. Backup Archive Contents

A complete backup of the v3.0 Final Prototype is stored in the `backups/final-prototype-20260505/` directory within the project. This directory contains compressed archives of the source code, public assets, the database file, and all configuration files needed for restoration. Additionally, the git repository contains the full commit history and the `v3.0-final-prototype` tag for source-level recovery.

File	Contents	Restoration Use
<code>custom.db</code>	Complete SQLite database with all user data, posts, follows, tokens, etc.	Copy to <code>db/custom.db</code> , update <code>DATABASE_URL</code>
<code>src-source.tar.gz</code>	Complete <code>src/</code> directory with all TypeScript/React source files	Extract to project root: <code>tar xzf src-source.tar.gz</code>
<code>public-assets.tar.gz</code>	Complete <code>public/</code> directory with logos, images, uploads	Extract to project root: <code>tar xzf public-assets.tar.gz</code>
<code>package.json</code>	Node.js dependencies with exact version ranges	<code>npm install</code> / <code>bun install</code> to restore <code>node_modules</code>
<code>bun.lock</code>	Lockfile for deterministic dependency resolution	Ensures identical dependency versions
<code>schema.prisma</code>	Complete Prisma database schema (20+ models)	Copy to <code>prisma/schema.prisma</code> , run <code>prisma generate</code>
<code>.env</code>	Environment variables (database URL, auth secrets)	Copy to project root
<code>next.config.ts</code>	Next.js configuration (standalone output, rewrites)	Copy to project root
<code>tailwind.config.ts</code>	Tailwind CSS configuration with custom theme	Copy to project root
<code>tsconfig.json</code>	TypeScript configuration with path aliases	Copy to project root
<code>Caddyfile</code>	Caddy reverse proxy configuration	Copy to project root, reload Caddy
<code>ecosystem.config.cjs</code>	PM2 process manager configuration	Copy to project root, run <code>pm2 start</code>
<code>start-server.sh</code>	Production server startup script	Copy to project root, <code>chmod +x</code> , execute
<code>postcss.config.mjs</code>	PostCSS configuration for Tailwind	Copy to project root
<code>components.json</code>	shadcn/ui component configuration	Copy to project root

Table 12: Backup Archive File Inventory

9. Full Restoration Procedure

This section provides the complete step-by-step procedure to restore the ORRA v3.0 Final Prototype from the backup archive to a fully running state. The procedure assumes a clean server environment with Node.js (v18+), npm or Bun, and Caddy installed. Each step is mandatory and must be executed in sequence. Verification steps are included at key milestones to confirm successful restoration.

9.1 Fresh Environment Setup

Step	Command	Verification
1	<code>mkdir -p /home/z/my-project && cd /home/z/my-project</code>	<code>pwd</code> shows correct path
2	Copy all backup files to <code>/home/z/my-project/</code>	<code>ls</code> shows <code>.env</code> , <code>package.json</code> , etc.
3	<code>tar xzf src-source.tar.gz</code>	<code>ls src/app/page.tsx</code> exists
4	<code>tar xzf public-assets.tar.gz</code>	<code>ls public/orra-logo.png</code> exists
5	<code>cp custom.db db/custom.db</code>	<code>ls db/custom.db</code> exists
6	<code>cp schema.prisma prisma/schema.prisma</code>	<code>ls prisma/schema.prisma</code> exists
7	<code>npm install</code>	<code>node_modules/</code> directory populated
8	<code>npx prisma generate</code>	No errors in output
9	<code>npx prisma db push</code>	Database tables created
10	<code>npm run build</code>	<code>.next/standalone/server.js</code> exists
11	<code>chmod +x start-server.sh && bash start-server.sh</code>	<code>curl localhost:3000</code> returns HTML
12	Configure Caddyfile, reload Caddy	<code>curl port 81</code> returns ORRA page

Table 13: Fresh Environment Restoration Steps

9.2 Git-Based Restoration

If the git repository is available, restoration is even simpler. Clone the repository, checkout the v3.0-final-prototype tag, install dependencies, and build. The git repository contains the complete source code including the backup directory. The database backup must still be manually restored from the backup file.

```
git clone /home/z/my-project
cd /home/z/my-project
```

```

git checkout v3.0-final-prototype
cp backups/final-prototype-20260505/custom.db db/custom.db
npm install
npx prisma generate
npx prisma db push
npm run build
bash start-server.sh

```

9.3 Post-Restoration Verification

After restoring the application, perform the following verification checks to ensure all features are working correctly. These checks confirm that the database, API routes, authentication, and UI components are all functioning as expected.

Check	Method	Expected Result
Server Running	<code>curl -s localhost:3000 head -5</code>	HTML response with ORRA content
Auth Login	<code>POST /api/auth/callback/credentials</code>	Session token returned
User Profile	<code>GET /api/me</code> with session cookie	JSON with user data and tokens
Feed Posts	<code>GET /api/posts</code>	Array of post objects
Database Integrity	<code>npx prisma studio</code>	All tables visible and populated
Static Assets	<code>GET /images/orra-logo.png</code>	PNG image returned (200 OK)
Settings Page	Navigate to Settings view	Holographic toggles visible
Daily Digest	Login for first time today	Digest modal auto-appears

Table 14: Post-Restoration Verification Checklist

10. Critical Patterns & Known Gotchas

Over the course of development, several non-obvious patterns and edge cases were discovered that can cause the application to malfunction if not handled correctly. This section documents these critical patterns so they are not lost during future rebuilds or modifications.

10.1 Static File Copying (CRITICAL)

The most common cause of "features not showing up" after a build is incorrect static file copying. The Next.js standalone build produces a `.next/standalone/` directory that does NOT include static assets or the

public directory. These must be manually copied after each build. The critical pattern is:

```
# CORRECT: Always clear target first, then copy fresh
rm -rf .next/standalone/.next/static/*
cp -r .next/static/* .next/standalone/.next/static/
rm -rf .next/standalone/public
cp -r public .next/standalone/public
```

NEVER use `cp -rn` (no-clobber) for static files. This flag skips existing files, leaving stale chunks from previous builds that cause the UI to render old or missing content. The `-rn` flag was responsible for multiple "feature not showing up" bugs during development.

10.2 Zustand localStorage Rehydration

The Zustand store uses a persist middleware that saves state to localStorage. On app load, the store rehydrates from localStorage before the component tree renders. This means that stale view state (e.g., `currentView: "settings"`) can persist across page reloads. The store includes a `resetStore` action and a `partialize` function that controls which fields are persisted. When adding new settings fields, they **MUST** be added to both the initial state and the `partialize` function, otherwise they will not survive page reloads.

10.3 NavView Routing Pattern

ORRA uses a client-side view routing pattern rather than Next.js file-based routing. The `currentView` state in the Zustand store determines which component renders in the MainContent area. When adding a new view (e.g., Settings), three places must be updated: (1) the `NavView` type in `orra-store.ts` must include the new view name, (2) the MainContent switch in `page.tsx` must have a case for the new view, and (3) the `sidebar.tsx` must include a navigation link that sets `currentView` to the new view name. Missing any of these three will result in the view being inaccessible or rendering a blank area.

10.4 Server Restart Sequence

After modifying source code and rebuilding, the old Node.js server process must be killed before starting a new one. Simply running `start-server.sh` again without killing the old process will result in a port conflict error. The `start-server.sh` script handles this automatically with `fuser -k 3000/tcp`, but if starting the server manually, always kill the old process first. A 1-second sleep after killing is recommended to ensure the port is fully released.

10.5 Prisma Schema and Database Sync

After any changes to `prisma/schema.prisma`, run `npx prisma db push` to sync the database schema. Unlike `prisma migrate`, `db push` does not create migration files and applies schema changes directly.

This is appropriate for the SQLite development database. However, be aware that destructive schema changes (e.g., removing a column) will result in data loss. Always backup the database before running `db push` after schema modifications. The backup at `backups/final-prototype-20260505/custom.db` represents the known-good state of the database.

10.6 Caddy Configuration

The Caddyfile in the project root defines a reverse proxy from port 81 to `localhost:3000`. If Caddy is running as a system service, the Caddyfile must be placed in the Caddy configuration directory (typically `/etc/caddy/`) and the service reloaded with `systemctl reload caddy`. The Caddyfile also supports a dynamic port query parameter (`XTransformPort`) for proxying to arbitrary local ports, which is useful for testing multiple services.

11. Test Account Credentials

The following test account is available in the database for verification purposes. This account has the founder role and includes all badge and token data needed for full feature testing. The account should be used for smoke testing after restoration to verify that authentication, profile rendering, and all interactive features are working correctly.

Field	Value
Email	<code>nickjoseph8087@gmail.com</code>
Password	<code>orra504</code>
Handle	<code>@nickorra</code>
Role	Founder (verified badge + exclusive CSS animations)

Table 15: Test Account Credentials

12. Git Reference

The ORRA project is maintained in a git repository with a complete commit history. The v3.0 Final Prototype is tagged with `v3.0-final-prototype` at commit `5517804`. This tag represents the exact state of the codebase that should be used for mainstream launch. All source files, configuration files, and the backup directory are committed under this tag. The git history also includes earlier milestones (`v2.5`, `v2.5.1`) for reference.

Reference	Value
-----------	-------

Tag	v3.0-final-prototype
Commit SHA	5517804
Commit Message	ORRA Final Prototype v3.0 - Settings, Daily Digest, Holographic Toggles, Locked Build
Branch	main
Previous Tag	v2.5.1 (ORRA v2.5.1 - Story viewer fix)

Table 16: Git Reference Information

13. Quick Restore from Git Tag

For the fastest possible restoration, use the git tag to checkout the exact state of the v3.0 Final Prototype. The following one-liner sequence restores the complete application to a running state, assuming Node.js (v18+) and npm are already installed on the target machine. This is the recommended restoration method when the git repository is accessible.

```
# Clone and checkout the final prototype tag
git clone /home/z/my-project && cd /home/z/my-project
git checkout v3.0-final-prototype

# Restore database from backup
mkdir -p db
cp backups/final-prototype-20260505/custom.db db/custom.db

# Install dependencies and build
npm install
npx prisma generate
npm run build

# Start the server
chmod +x start-server.sh && bash start-server.sh
```

If the backup directory is not available in the git repository, the database can be recreated from the Prisma schema using `npx prisma db push`. However, this will result in an empty database without the seed data, test accounts, and sample posts. The seed script at `prisma/seed.ts` can be used to populate the database with initial data by running `npm run db:seed` after `db push`.